
AIOps Lab 1: Observability Engineering Report

Monitoring Structured Logs, RED Metrics, and Anomaly Injection

Basant Awad - 22101405
March 9, 2026

LOG SCHEMA DESIGN

The structured JSON schema was designed to facilitate automated triaging by providing high-context data points that are easily indexable.

- **Timestamp & Severity:** Essential for time-series alignment and basic filtering/alerting.
- **Request ID:** Acts as a correlation ID to link disparate logs to a single request trace for deep-dive analysis.
- **Latency (ms):** Required for performance profiling and identifying "slow-burn" regressions.
- **Error Category:** Uses low-cardinality labels (`DATABASE_ERROR`, `SYSTEM_ERROR`, `TIMEOUT_ERROR`) to allow ML models to classify root causes without expensive natural language processing of stack traces.
- **Build Version:** Enables "Point-in-Time" analysis to correlate spikes with specific code deployments.
- **HTTP Context (Method, Path, IP, User Agent, Sizes):** Provides standard web telemetry to identify exactly which endpoint was hit, the bandwidth used, and the client origin factors.

METRICS DESIGN (RED PATTERN)

We implemented Prometheus RED metrics to provide aggregate system health visibility at a glance.

- **Labels (method, path, status, error_category):** Chosen to provide per-endpoint granularity. High-cardinality labels (like user IDs or raw IPs) were avoided to prevent Prometheus storage bloat and slow query performance.
- **Duration Buckets:** Defined as `[0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10]`. Small increments capture fast API responses, while the 10s ceiling effectively monitors hard timeouts and SLA breaches (e.g., the intentional 5-7s delays on the slow endpoint).

ANOMALY INJECTION & SIMULATION

To test the observability pipeline, a synthetic "Error Spike" was injected via a traffic generator.

- **Control:** The anomaly was strictly limited to a 2-minute window to simulate a transient failure and test the system's recovery capabilities.
- **Visibility:** To ensure the anomaly was statistically significant for monitoring, the traffic weight for the `/api/error` endpoint was increased from 5% to 40%. This creates a clear "Step-Up" pattern in the metrics as seen in the dashboards.

DASHBOARD EVIDENCE & ANALYSIS

Full Monitoring Overview

Traffic and Error Breakdown

As shown in Figure 2 and 3, the anomaly is clearly visible as a sharp spike in both Error RPS and Error Percentage.

CONCLUSION

The lab implementation successfully bridges structured logging with aggregate metrics. The dashboard evidence confirms that the engineered anomaly was captured with high precision, demonstrating the stack's readiness for AIOps-driven incident management.

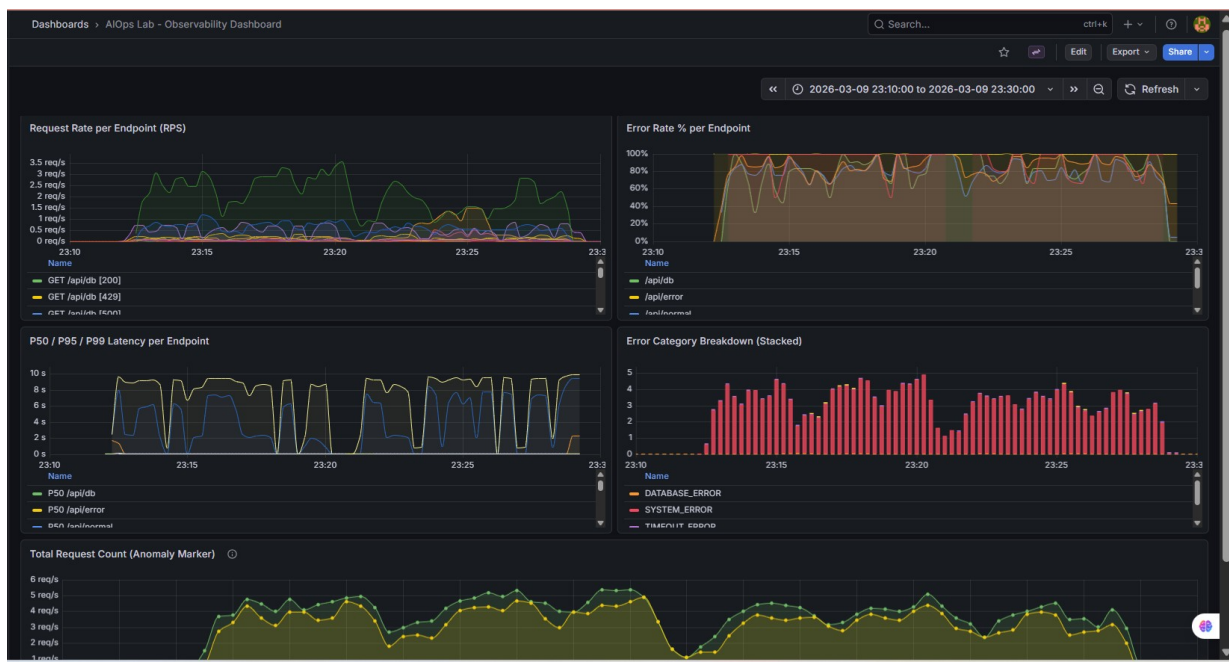


Figure 1: Full dashboard showing the annotated 2-minute anomaly window.

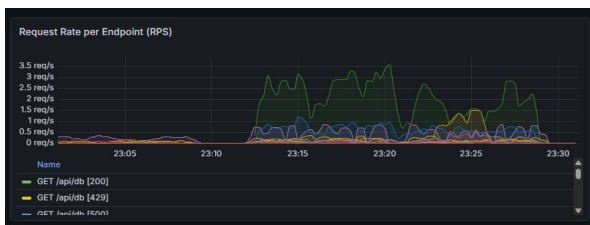


Figure 2: Request Rate per Endpoint



Figure 3: Error Rate Percentage

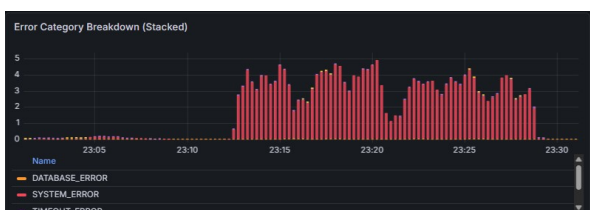


Figure 4: Error Category Breakdown



Figure 5: Total Request Count Anomaly